Proyecto Final

Verificanet

ÍNDICE

1. INTRODUCCIÓN **3**
1.1 Descripción del Proyecto 3
1.2 Objetivos 4
1.3 Tecnologías Utilizadas 4
2. ARQUITECTURA DEL SISTEMA **5**
2.1 Diagrama de Red 5
2.2 Subredes y Segmentación 5
2.3 Flujo de Tráfico 6
2.4 Puertos y Protocolos 6
3. BASE DE DATOS **7**
3.1 Modelo Entidad-Relación 7
3.2 Tablas de la Base de Datos 7
3.3 Usuarios de Acceso a la Base de Datos 10
4. INFRAESTRUCTURA LINUX **10**
4.1 Vagrant - Automatización del Despliegue 10
Comandos Vagrant útiles 11
4.2 Firewall (192.168.50.1 / 192.168.60.1) 11
Características clave 11
4.3 Web - Balanceador Nginx (192.168.50.30) 12
Configuración Nginx 12
4.4 Backends (192.168.50.41-42 / 192.168.60.41-42) 13
Configuración dual-homed (dos interfaces) 13
2. Login 14
3. Listado de Incidencias 14
4. Estadísticas 15
Verificación 15
4.5 Database (192.168.60.50) 15
5. WINDOWS SERVER **16**
5.1 Active Directory - Dominio verificanet.local 16
5.2 Estructura de OUs (Unidades Organizativas) 17
5.3 Usuarios y Grupos 17
Grupos de Seguridad 17
5.4 DNS Server 19

Zonas DNS Configuradas 19
5.5 DHCP Server 19
5.6 Servidor FTP 21
Características 21
Usuarios FTP autorizados 21
6. PANEL DE ADMINISTRACIÓN PYTHON 22
6.1 Descripción 22
6.2 Requisitos 22
6.3 Configuración SSH sin contraseña 22
6.4 Funcionalidades 23
6.5 Ejecución 23
7. SEGURIDAD 24
7.1 Segmentación de Red 24
7.2 Control de Acceso 25
7.3 Firewall iptables 25
7.4 Buenas Prácticas Implementadas 26
8. INSTALACIÓN Y CONFIGURACIÓN 26
8.1 Requisitos Previos 26
8.2 Instalación Paso a Paso 26
8.3 Verificación Final 29
Comandos de Verificación 29
9. APÉNDICES 29
9.1 Tabla de IPs Completa 29
9.2 Credenciales del Sistema 29
9.3 Puertos de Red 30
9.4 Comandos Útiles 30
9.5 Solución de Problemas 31
CONCLUSIÓN 33

1. INTRODUCCIÓN

1.1 Descripción del Proyecto

Verificanet es un sistema integral de gestión de incidencias diseñado para empresas de servicios IT que requieren:

- Gestión centralizada de clientes y contratos
- Seguimiento de incidencias técnicas
- Control de acceso basado en roles (RBAC)
- Alta disponibilidad mediante balanceo de carga
- Seguridad mediante segmentación de red (DMZ + Red Interna)

1.2 Objetivos

- **Disponibilidad:** Sistema operativo 24/7 con redundancia
- **Seguridad:** Implementación de DMZ y políticas de firewall estrictas
- **Escalabilidad:** Arquitectura preparada para crecer
- **Gestión centralizada:** Active Directory para usuarios y equipos
- **Monitorización:** Panel de administración remota

1.3 Tecnologías Utilizadas

Virtualización e Infraestructura

- **Vagrant** - Automatización de despliegue
- **VirtualBox** - Plataforma de virtualización
- **Debian/Ubuntu Server** - Sistemas operativos Linux

Servicios Linux

- **Nginx** - Balanceador de carga y proxy inverso
- **Apache + PHP** - Servidor de aplicaciones (backends)
- **MariaDB** - Sistema gestor de base de datos
- **iptables** - Firewall y NAT

Servicios Windows

- **Windows Server 2019/2022** - Sistema operativo
- **Active Directory** - Directorio centralizado
- **DNS Server** - Resolución de nombres
- **DHCP Server** - Asignación automática de IPs
- **IIS + FTP** - Servidor FTP con autenticación AD

Aplicaciones

- **PHP** - API REST para backend
- **Python + tkinter** - Panel de administración gráfico
- **SSH** - Administración remota segura

2. ARQUITECTURA DEL SISTEMA

2.1 Diagrama de Red

```
                INTERNET
                    |
                [Firewall]
               192.168.50.1
          192.168.60.1 (Red Interna)
                    |
      ┌──────────────────────────────────────┐
      │                                      │
      │                                      │
  DMZ (192.168.50.0/24)    Red Interna (192.168.60.0/24)
      │                                      │
  ┌───┴──────────────────────────────┐   ┌──┴──────────┐
  │   │   │   │   │   │   │
[Web] [Back1] [Back2] [Win] [Back1] [Back2] [Database]
.30   .41     .42     .10   .41     .42     .50
```

2.2 Subredes y Segmentación

DMZ (Zona Desmilitarizada) - 192.168.50.0/24

Propósito: Servicios accesibles desde Internet

Host	IP	Función	SO
Firewall	192.168.50.1	Router/Firewall/NAT	Debian 11
Windows Server	192.168.50.10	AD/DNS/DHCP/FTP	Windows Server
Clientes Windows	192.168.50.21-200	Estaciones de trabajo	Windows 10/11 Pro
Web (Nginx)	192.168.50.30	Balanceador de carga	Ubuntu 22.04
Backend1 (eth0)	192.168.50.41	Servidor API (interfaz DMZ)	Ubuntu 22.04
Backend2 (eth0)	192.168.50.42	Servidor API (interfaz DMZ)	Ubuntu 22.04

Red Interna - 192.168.60.0/24

Propósito: Servicios críticos protegidos (solo acceso desde backends)

Host	IP	Función	SO
Firewall	192.168.60.1	Gateway red interna	Debian 11
Backend1 (eth1)	192.168.60.41	Servidor API (interfaz interna)	Ubuntu 22.04
Backend2 (eth1)	192.168.60.42	Servidor API (interfaz interna)	Ubuntu 22.04
Database	192.168.60.50	Servidor de base de datos	Ubuntu 22.04

2.3 Flujo de Tráfico

Escenario 1: Usuario Web

Usuario → Firewall (NAT) → Web (Nginx) → Backend1/2 (DMZ) →
Backend1/2 (Interna) → Database → Respuesta

Escenario 2: Administrador Windows

Admin Windows → Firewall → Servidores Linux (SSH) → Gestión

Escenario 3: Cliente Windows (sin permisos)

Cliente Windows → Firewall → [BLOQUEADO a Red Interna]
Cliente Windows → Firewall → Internet [PERMITIDO]

2.4 Puertos y Protocolos

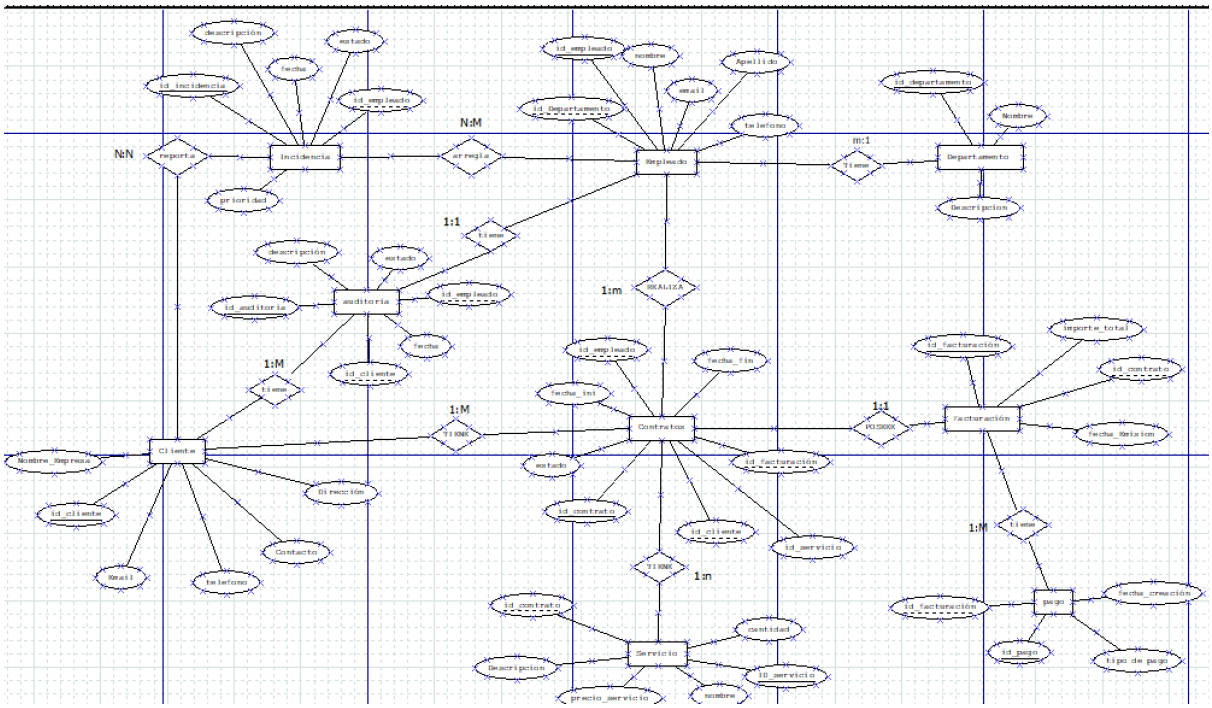
Servicio	Puerto	Protocolo	Origen	Destino
HTTP	80	TCP	Internet	Web (192.168.50.30)
HTTP	80	TCP	Web	Backends

				(192.168.50.41-42)
MySQL	3306	TCP	Backends (192.168.60.41-42)	Database (192.168.60.50)
SSH	22	TCP	Admin Windows	Todos los Linux
DNS	53	UDP/TCP	Todos	Windows Server (192.168.50.10)
DHCP	67-68	UDP	Clientes	Windows Server
FTP	21	TCP	Usuarios autorizados	Windows Server

---

3. BASE DE DATOS

3.1 Modelo Entidad-Relación



3.2 Tablas de la Base de Datos

Tabla: departamentos

```
CREATE TABLE departamentos (
    id_departamento INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT
);
```

Registros:

1. Soporte Técnico
2. Ventas
3. Administración
4. Desarrollo
5. Auditoría

Tabla: empleados

```
CREATE TABLE empleados (
   id_empleado INT AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL,
   apellido VARCHAR(100) NOT NULL,
   email VARCHAR(100) UNIQUE NOT NULL,
   telefono VARCHAR(20),
   id_departamento INT,
   fecha_contratacion DATE,
   FOREIGN KEY (id_departamento) REFERENCES departamentos(id_departamento)
);
```

Empleados del sistema:

- María González (Soporte Técnico)
- Juan Martínez (Soporte Técnico)
- Ana Rodríguez (Administración)
- Carlos López (Desarrollo)
- Laura Fernández (Ventas)
- Pedro Sánchez (Auditoría)

Tabla: clientes

```
CREATE TABLE clientes (
   id_cliente INT AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(100),
   apellido VARCHAR(100),
   nombre_empresa VARCHAR(200),
   email VARCHAR(100) UNIQUE NOT NULL,
   telefono VARCHAR(20),
   direccion TEXT,
   tipo_cliente ENUM('particular', 'empresa') DEFAULT 'particular',
   fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Tabla: servicios

```
CREATE TABLE servicios (
   id_servicio INT AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL,
```

```
    descripcion TEXT,
    precio_base DECIMAL(10,2),
    tipo_servicio ENUM('soporte', 'mantenimiento', 'desarrollo', 'consultoría')
);
```

Servicios ofrecidos:

1. Soporte Técnico Básico (299.99€)
2. Mantenimiento Preventivo (499.99€)
3. Desarrollo Web (2999.99€)
4. Consultoría IT (1499.99€)
5. Auditoría de Seguridad (1999.99€)
6. Backup Cloud (199.99€)

Tabla: contratos

```
CREATE TABLE contratos (
    id_contrato INT AUTO_INCREMENT PRIMARY KEY,
    id_cliente INT,
    id_servicio INT,
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE,
    estado ENUM('activo', 'inactivo', 'finalizado') DEFAULT 'activo',
    notas TEXT,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) ON DELETE CASCADE,
    FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)
);
```

Tabla: incidencias

```
CREATE TABLE incidencias (
    id_incidencia INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(200) NOT NULL,
    descripcion TEXT,
    estado ENUM('reportada', 'en_proceso', 'resuelta', 'cerrada') DEFAULT 'reportada',
    prioridad ENUM('baja', 'media', 'alta', 'critica') DEFAULT 'media',
    id_cliente INT,
    id_empleado_asignado INT,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    fecha_resolucion DATETIME,
    notas_resolucion TEXT,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) ON DELETE CASCADE,
    FOREIGN KEY (id_empleado_asignado) REFERENCES empleados(id_empleado)
);
```

Tabla: usuarios

```
CREATE TABLE usuarios (
   id_usuario INT AUTO_INCREMENT PRIMARY KEY,
   username VARCHAR(50) UNIQUE NOT NULL,
   password VARCHAR(255) NOT NULL,
   rol ENUM('admin', 'empleado', 'cliente') NOT NULL,
   id_empleado INT NULL,
   id_cliente INT NULL,
   activo BOOLEAN DEFAULT TRUE,
   fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado) ON DELETE
CASCADE,
   FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) ON DELETE CASCADE
);
```

3.3 Usuarios de Acceso a la Base de Datos

Usuario: verificanet_user@192.168.60.41

Permisos: SELECT, INSERT, UPDATE, DELETE
Base de datos: verificanet_servicios
Contraseña: Verific@2024!

Usuario: verificanet_user@192.168.60.42

Permisos: SELECT, INSERT, UPDATE, DELETE
Base de datos: verificanet_servicios
Contraseña: Verific@2024!

Nota de seguridad: El acceso está restringido por IP mediante iptables adicionales en el servidor de base de datos.

---

4. INFRAESTRUCTURA LINUX

4.1 Vagrant - Automatización del Despliegue

Archivo: Vagrantfile

El proyecto utiliza Vagrant para automatizar el despliegue de 5 máquinas virtuales.

Configuración de VMs:

- firewall (Debian 11) 512MB RAM, 1 CPU
- web (Ubuntu 22) 512MB RAM, 1 CPU
- backend1 (Ubuntu 22) 512MB RAM, 1 CPU
- backend2 (Ubuntu 22) 512MB RAM, 1 CPU
- database (Ubuntu 22) 1024MB RAM, 1 CPU

Redes configuradas:

- Red DMZ: `virtualbox__intnet: "dmz"`
- Red Interna: `virtualbox__intnet: "interna"`

Port Forwarding:

- Host: 8080 → Web: 80 (acceso al balanceador desde el PC)

Comandos Vagrant útiles

```
# Levantar toda la infraestructura
vagrant up

# Levantar una VM específica
vagrant up firewall

# Ver estado de las VMs
vagrant status

# Conectarse por SSH
vagrant ssh web

# Reiniciar una VM
vagrant reload web

# Destruir y recrear
vagrant destroy -f
vagrant up

# Detener todas las VMs
vagrant halt

# Aplicar cambios de aprovisionamiento
vagrant provision backend1
```

4.2 Firewall (192.168.50.1 / 192.168.60.1)

Función

Router central con NAT, firewall y enrutamiento entre DMZ y Red Interna.

Características clave

1. **NAT hacia Internet** (enp0s3 - interfaz NAT de Vagrant)

2. **Interfaz DMZ** (enp0s8 - 192.168.50.1)
3. **Interfaz Red Interna** (enp0s9 - 192.168.60.1)
4. **Políticas DROP por defecto** (seguridad máxima)
5. **Logging de conexiones bloqueadas**

Reglas iptables principales

```
# NAT hacia Internet
iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE

# Políticas por defecto
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT

# SSH permitido desde ambas redes
iptables -A INPUT -p tcp --dport 22 -s 192.168.50.0/24 -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -s 192.168.60.0/24 -j ACCEPT

# Web (192.168.50.30) puede recibir tráfico desde Internet
iptables -A FORWARD -d 192.168.50.30 -j ACCEPT

# Web → Backends (HTTP)
iptables -A FORWARD -s 192.168.50.30 -d 192.168.50.41 -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -s 192.168.50.30 -d 192.168.50.42 -p tcp --dport 80 -j ACCEPT

# Backends → Base de datos (MySQL)
iptables -A FORWARD -s 192.168.60.41 -d 192.168.60.50 -p tcp --dport 3306 -j ACCEPT
iptables -A FORWARD -s 192.168.60.42 -d 192.168.60.50 -p tcp --dport 3306 -j ACCEPT

# Ruteo entre DMZ ↔ Red Interna
iptables -A FORWARD -i enp0s8 -o enp0s9 -j ACCEPT
iptables -A FORWARD -i enp0s9 -o enp0s8 -j ACCEPT

# Logging
iptables -A FORWARD -j LOG --log-prefix "FW-BLOCKED: "
```

Verificación

```
# Ver reglas activas
sudo iptables -L -n -v

# Ver reglas NAT
sudo iptables -t nat -L -n -v

# Ver logs del firewall
sudo tail -f /var/log/syslog | grep FW-BLOCKED
```

4.3 Web - Balanceador Nginx (192.168.50.30)

Función

Balanceador de carga HTTP que distribuye peticiones entre backend1 y backend2.

Configuración Nginx

Archivo: `/etc/nginx/sites-available/default`

```
upstream backends {
    server 192.168.50.41;
    server 192.168.50.42;
}

server {
    listen 80;
    server_name _;

    access_log /var/log/nginx/balanceador_access.log;
    error_log  /var/log/nginx/balanceador_error.log;

    location / {
        proxy_pass http://backends;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location /health {
        return 200 "OK - Balanceador activo\n";
        add_header Content-Type text/plain;
    }
}
```

Logs importantes

```
# Accesos al balanceador
sudo tail -f /var/log/nginx/balanceador_access.log

# Errores del balanceador
sudo tail -f /var/log/nginx/balanceador_error.log
```

Pruebas de funcionamiento

```
# Desde el firewall o cualquier VM en DMZ
curl http://192.168.50.30/health
# Respuesta: OK - Balanceador activo

# Ver a qué backend se redirige (múltiples peticiones)
for i in {1..10}; do curl http://192.168.50.30/; done
```

4.4 Backends (192.168.50.41-42 / 192.168.60.41-42)

Función

Servidores de aplicación con Apache + PHP que ejecutan la API REST.

Configuración dual-homed (dos interfaces)

Cada backend tiene **DOS interfaces de red**:

1. **Interfaz DMZ** (eth0): Recibe peticiones del balanceador
    - Backend1: 192.168.50.41
    - Backend2: 192.168.50.42
2. **Interfaz Red Interna** (eth1): Conecta con la base de datos
    - Backend1: 192.168.60.41
    - Backend2: 192.168.60.42

API REST - Endpoints

Archivo: `/var/www/html/api.php`

1. Health Check

GET /api.php?action=health

Respuesta:

```
{
  "status": "ok",
  "database": "connected"
}
```

2. Login

POST /api.php?action=login
Parámetros: username, password

Respuesta exitosa:

```
{
  "success": true,
  "user": {
    "id_usuario": 1,
    "username": "admin",
    "rol": "admin"
```

```
  }
}
```

3. Listado de Incidencias

GET /api.php?action=incidencias

Respuesta:

```
[
  {
    "id_incidencia": 1,
    "titulo": "Error en conexión VPN",
    "descripcion": "No se puede conectar a la VPN corporativa",
    "estado": "en_proceso",
    "prioridad": "alta",
    "cliente_nombre": "Empresa Test S.L.",
    "empleado_nombre": "María González"
  },
  ...
]
```

4. Estadísticas

GET /api.php?action=stats

Respuesta:

```
{
  "total_clientes": 2,
  "total_incidencias": 6,
  "incidencias_abiertas": 3
}
```

Verificación

```
# Conectar al backend
vagrant ssh backend1

# Probar la API localmente
curl http://localhost/api.php?action=health
curl http://localhost/api.php?action=stats

# Ver logs de Apache
sudo tail -f /var/log/apache2/access.log
sudo tail -f /var/log/apache2/error.log
```

4.5 Database (192.168.60.50)

Función

Servidor MariaDB alojando la base de datos `verificanet_servicios`.

Seguridad implementada

Escucha solo en Red Interna (192.168.60.50)

bind-address = 192.168.60.50

1. **Usuarios con restricción por IP:**
   - `verificanet_user@192.168.60.41`
   - `verificanet_user@192.168.60.42`

Firewall iptables adicional:

```
# Solo backends pueden conectar al puerto 3306
iptables -A INPUT -p tcp --dport 3306 -s 192.168.60.41 -j ACCEPT
iptables -A INPUT -p tcp --dport 3306 -s 192.168.60.42 -j ACCEPT
iptables -A INPUT -p tcp --dport 3306 -j DROP
```

2. **Verificación**

```
# Conectar a la VM
vagrant ssh database

# Acceder a MySQL
sudo mysql -u root

# Ver bases de datos
SHOW DATABASES;

# Usar la BD del proyecto
USE verificanet_servicios;

# Ver tablas
SHOW TABLES;

# Ver usuarios
SELECT User, Host FROM mysql.user;

# Probar conexión desde backend1
mysql -h 192.168.60.50 -u verificanet_user -p'Verific@2024!' verificanet_servicios
```

---

5. WINDOWS SERVER

5.1 Active Directory - Dominio verificanet.local

Instalación del Dominio

Script: `Active_Directory.ps1` (Instalación inicial de AD)

```
# Configuración
$dominio = "verificanet.local"
$netbios = "VERIFICANET"
$passwordDSRM = "Admin123!"

# Instalar AD-Domain-Services
Install-WindowsFeature -Name AD-Domain-Services -IncludeManagementTools

# Promover a Domain Controller
Install-ADDSForest `
   -DomainName $dominio `
   -DomainNetbiosName $netbios `
   -ForestMode "WinThreshold" `
   -DomainMode "WinThreshold" `
   -InstallDns:$true `
   -SafeModeAdministratorPassword $safePwd `
   -Force:$true
```

Resultado: El servidor se reinicia automáticamente y queda como Domain Controller.

5.2 Estructura de OUs (Unidades Organizativas)

Script: `OUS.ps1`

```
DC=verificanet,DC=local
│
─ OU=VerificaNet
 │_
   ├─ OU=Usuarios
   │   ├─ OU=SoporteTecnico
   │   ├─ OU=Ventas
   │   ├─ OU=Administracion
   │   ├─ OU=Sistemas
   │   └─ OU=Auditoria
   │
   ├─ OU=Grupos
   │
   ├─ OU=Equipos
   │   ├─ OU=Servidores
   │   ├─ OU=Impresoras
   │   ├─ OU=SoporteTecnico
   │   ├─ OU=Ventas
   │   ├─ OU=Administracion
   │   ├─ OU=Sistemas
   │   └─ OU=Auditoria
   │_____________________________
```

Creación de OUs

```
# OU principal
New-ADOrganizationalUnit -Name "VerificaNet" -Path "DC=verificanet,DC=local"

# OUs de usuarios
New-ADOrganizationalUnit -Name "Usuarios" -Path "OU=VerificaNet,DC=verificanet,DC=local"
New-ADOrganizationalUnit -Name "SoporteTecnico" -Path
"OU=Usuarios,OU=VerificaNet,DC=verificanet,DC=local"

# OUs de equipos
New-ADOrganizationalUnit -Name "Equipos" -Path "OU=VerificaNet,DC=verificanet,DC=local"
New-ADOrganizationalUnit -Name "SoporteTecnico" -Path
"OU=Equipos,OU=VerificaNet,DC=verificanet,DC=local"
```

5.3 Usuarios y Grupos

Script: `Usuarios-grupos.ps1`

Grupos de Seguridad

Grupo	Descripción	Miembros
GRP_SoporteTecnico	Empleados de Soporte	mgonzalez, jmartinez
GRP_Ventas	Equipo comercial	lfernandez, storres
GRP_Administracion	Personal administrativo	arodriguez, mnavarro
GRP_Sistemas	Administradores de sistemas	clopez, adminvnet
GRP_Auditoria	Auditores	psanchez
GRP_Todos_Empleados	Todos los empleados	Todos los anteriores
GRP_FTP_Acceso	Acceso al servidor FTP	Todos los empleados

Usuarios del Dominio

Username	Nombre Completo	Departamento	OU	Contraseña
adminvnet	Admin VerificaNet	Sistemas	OU=Sistemas	Verific@2024!
mgonzalez	María González	Soporte Técnico	OU=SoporteTecnico	Verific@2024!
jmartinez	Juan Martínez	Soporte Técnico	OU=SoporteTecnico	Verific@2024!

arodriguez	Ana Rodríguez	Administración	OU=Administracion	Verific@2024!
mnavarro	Miguel Navarro	Administración	OU=Administracion	Verific@2024!
clopez	Carlos López	Sistemas	OU=Sistemas	Verific@2024!
lfernandez	Laura Fernández	Ventas	OU=Ventas	Verific@2024!
storres	Sofía Torres	Ventas	OU=Ventas	Verific@2024!
psanchez	Pedro Sánchez	Auditoría	OU=Auditoria	Verific@2024!

Creación de Usuarios (ejemplo)

```
New-ADUser `
    -GivenName "María" `
    -Surname "González" `
    -Name "María González" `
    -SamAccountName "mgonzalez" `
    -UserPrincipalName "mgonzalez@verificanet.local" `
    -EmailAddress "mgonzalez@verificanet.local" `
    -Department "SoporteTecnico" `
    -Title "Técnico de Soporte" `
    -Path "OU=SoporteTecnico,OU=Usuarios,OU=VerificaNet,DC=verificanet,DC=local" `
    -AccountPassword (ConvertTo-SecureString "Verific@2024!" -AsPlainText -Force) `
    -Enabled $true `
    -ChangePasswordAtLogon $false
```

5.4 DNS Server

Script: `DNS.ps1`

Zonas DNS Configuradas

Zona Directa: verificanet.local

Registro	Tipo	IP
dc1.verificanet.local	A	192.168.50.10
web.verificanet.local	A	192.168.50.30
backend1.verificanet.local	A	192.168.50.41
backend2.verificanet.local	A	192.168.50.42
firewall.verificanet.local	A	192.168.50.1

Zona Inversa: 50.168.192.in-addr.arpa

Resolución inversa para la DMZ (192.168.50.0/24).

Configuración DNS

```
# Crear zona directa
Add-DnsServerPrimaryZone -Name "verificanet.local" -ReplicationScope "Forest"

# Añadir registros A
Add-DnsServerResourceRecordA -Name "web" -ZoneName "verificanet.local" -IPv4Address "192.168.50.30"

# Crear zona inversa
Add-DnsServerPrimaryZone -NetworkID "192.168.50.0/24" -ReplicationScope "Forest"
```

Verificación

```
# Ver zonas DNS
Get-DnsServerZone

# Probar resolución directa
nslookup web.verificanet.local

# Probar resolución inversa
nslookup 192.168.50.30
```

5.5 DHCP Server

Script: `DHCP.ps1`

Configuración del Servidor DHCP

Ámbito (Scope): DMZ - 192.168.50.0/24

Parámetro	Valor
Nombre	Ambito-Verificanet-DMZ
Rango de IPs	192.168.50.21 - 192.168.50.200
Máscara de subred	255.255.255.0
Puerta de enlace	192.168.50.1 (Firewall)
DNS	192.168.50.10 (Windows Server)
Dominio DNS	verificanet.local

Reservas DHCP

Nombre	MAC Address	IP Reservada
PC-SOPORTE-01	08-00-27-XX-XX-01	192.168.50.21
PC-ADMIN-01	08-00-27-XX-XX-02	192.168.50.22
PC-VENTAS-01	08-00-27-XX-XX-03	192.168.50.23
PC-GENERAL-01	08-00-27-XX-XX-04	192.168.50.24

Creación del Ámbito

```
Add-DhcpServerv4Scope `
   -Name "Ambito-Verificanet-DMZ" `
   -StartRange 192.168.50.21 `
   -EndRange 192.168.50.200 `
   -SubnetMask 255.255.255.0 `
   -State Active

Set-DhcpServerv4OptionValue `
   -ScopeId 192.168.50.0 `
   -Router 192.168.50.1 `
   -DnsServer 192.168.50.10 `
   -DnsDomain "verificanet.local"
```

Verificación

```
# Ver ámbitos configurados
Get-DhcpServerv4Scope

# Ver reservas
Get-DhcpServerv4Reservation -ScopeId 192.168.50.0

# Ver concesiones activas
Get-DhcpServerv4Lease -ScopeId 192.168.50.0
```

5.6 Servidor FTP

Script: `ftp.ps1`

Características

- **Puerto:** 21 (FTP estándar)
- **Autenticación:** Active Directory
- **Carpetas por departamento:** Control de acceso mediante grupos de AD
- **SSL/TLS:** Opcional (no configurado en versión base)

Estructura de Carpetas

```
C:\FTP\
│
├─ SoporteTecnico\     (GRP_SoporteTecnico - Lectura/Escritura)
├─ Ventas\             (GRP_Ventas - Lectura/Escritura)
├─ Administracion\     (GRP_Administracion - Lectura/Escritura)
├─ Sistemas\           (GRP_Sistemas - Lectura/Escritura)
├─ Auditoria\          (GRP_Auditoria - Lectura/Escritura)
└─ Compartido\         (GRP_Todos_Empleados - Lectura)
```

Permisos NTFS

```
# Ejemplo: Carpeta de Soporte Técnico
$acl = Get-Acl "C:\FTP\SoporteTecnico"
$acl.SetAccessRuleProtection($true, $false)

$rule = New-Object System.Security.AccessControl.FileSystemAccessRule(
    "VERIFICANET\GRP_SoporteTecnico",
    "Modify",
    "ContainerInherit,ObjectInherit",
    "None",
    "Allow"
)

$acl.AddAccessRule($rule)
Set-Acl "C:\FTP\SoporteTecnico" $acl
```

Usuarios FTP autorizados

Todos los usuarios del dominio en `GRP_FTP_Acceso` pueden conectarse al FTP.

Conexión:

ftp://192.168.50.10
Usuario: VERIFICANET\mgonzalez
Contraseña: Verific@2024!

---

6. PANEL DE ADMINISTRACIÓN PYTHON

6.1 Descripción

Archivo: `admin_logs2.py`

Aplicación gráfica desarrollada en Python con tkinter que permite a administradores:

- Consultar logs de Nginx (errores y accesos)
- Consultar logs del sistema (syslog)
- Ver estado del servicio Nginx
- Reiniciar Nginx remotamente
- Todo mediante conexión SSH al servidor web (192.168.50.30)

6.2 Requisitos

En Windows Server/Cliente:

- Python 3.8 o superior
- Cliente SSH (OpenSSH Client)
- Conectividad de red a 192.168.50.30

En el servidor web (192.168.50.30):

- Servidor SSH (openssh-server)
- Usuario con privilegios sudo (vagrant)
- Nginx instalado

6.3 Configuración SSH sin contraseña

Para evitar pedir contraseña cada vez, configurar clave SSH:

```
# En Windows (PowerShell como administrador)
# Generar par de claves
ssh-keygen -t rsa -b 4096 -N "" -f C:\Users\clopez\.ssh\id_rsa

# Copiar clave pública al servidor
type C:\Users\clopez\.ssh\id_rsa.pub | ssh vagrant@192.168.50.30 "mkdir -p ~/.ssh && cat >>
~/.ssh/authorized_keys"
```

6.4 Funcionalidades

1. Ver errores de Nginx

```
def ver_errores():
    salida = ejecutar_ssh("sudo tail -n 50 /var/log/nginx/error.log")
    mostrar_en_pantalla("ERRORES NGINX - Últimas 50 líneas", salida)
```

2. Ver accesos web

```
def ver_accesos():
    salida = ejecutar_ssh("sudo tail -n 50 /var/log/nginx/access.log")
    mostrar_en_pantalla("ACCESOS WEB - Últimas 50 líneas", salida)
```

3. Ver log del sistema

```
def ver_syslog():
    salida = ejecutar_ssh("sudo tail -n 50 /var/log/syslog")
    mostrar_en_pantalla("LOG DEL SISTEMA - Últimas 50 líneas", salida)
```

4. Ver estado de Nginx

```
def ver_estado():
    salida = ejecutar_ssh("sudo systemctl status nginx")
    mostrar_en_pantalla("ESTADO DE NGINX", salida)
```

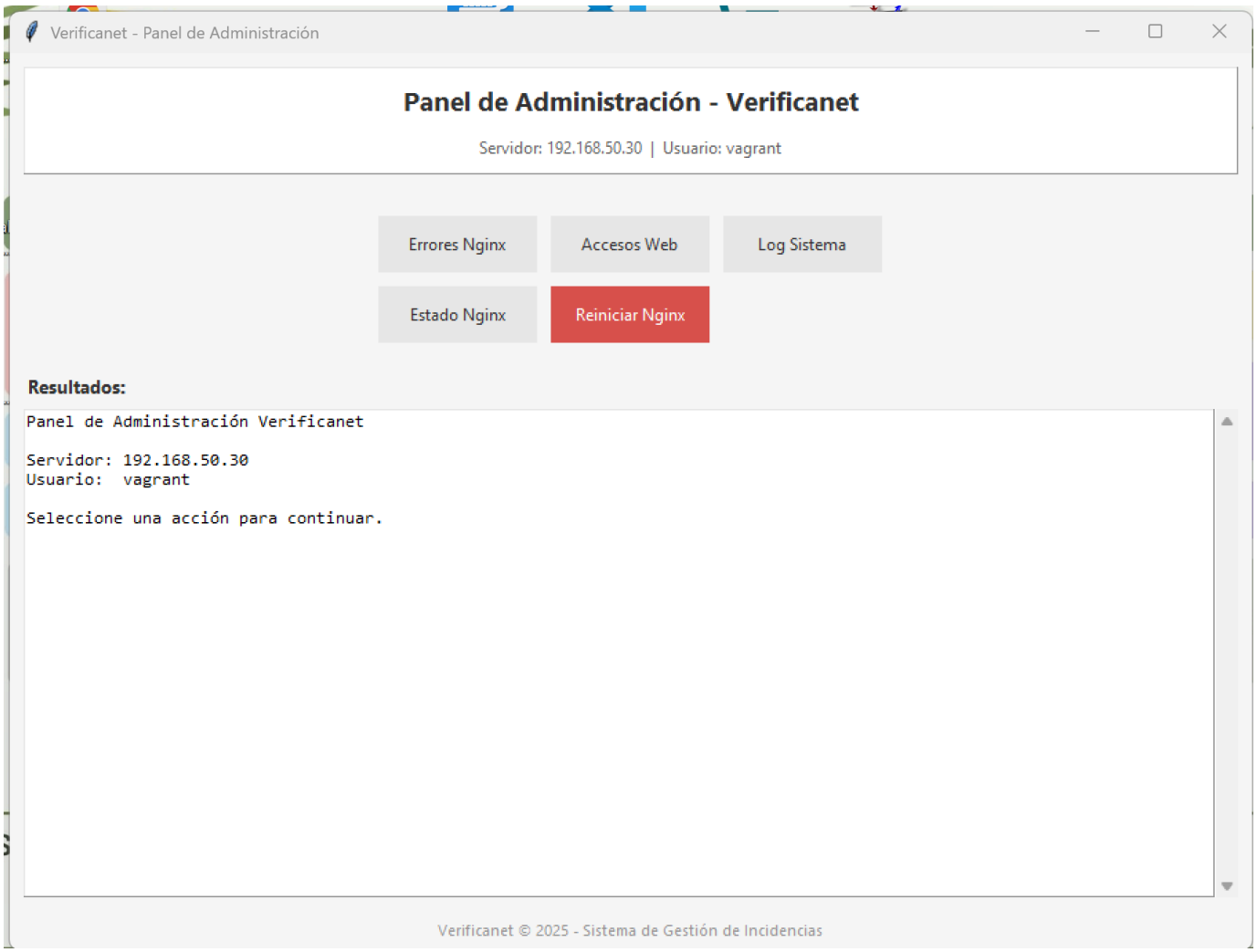
5. Reiniciar Nginx

```
def reiniciar_nginx():
    confirmado = messagebox.askyesno("Confirmar reinicio", "¿Está seguro?")
    if confirmado:
        ejecutar_ssh("sudo systemctl restart nginx")
        estado = ejecutar_ssh("sudo systemctl is-active nginx").strip()
        if estado == "active":
            messagebox.showinfo("Éxito", "Nginx reiniciado correctamente.")
```

6.5 Ejecución

```
# En Windows, como usuario clopez (Administrador de Sistemas)
python C:\Scripts\admin_logs2.py
```

Captura de pantalla (diseño)



---

7. SEGURIDAD

7.1 Segmentación de Red

Principio: Defensa en Profundidad

1. **DMZ (Zona Desmilitarizada)**
    - Servicios expuestos a Internet (Web, Backends interfaz pública)
    - Acceso limitado a Red Interna
2. **Red Interna**
    - Base de datos completamente aislada de Internet
    - Solo accesible desde backends autorizados
    - Sin acceso directo desde DMZ
3. **Firewall centralizado**
    - Control total del tráfico entre zonas
    - Logging de conexiones bloqueadas
    - Políticas DROP por defecto

7.2 Control de Acceso

Active Directory

- **Usuarios organizados por departamento** (OUs)
- **Grupos de seguridad** para permisos
- **GPOs** para políticas de seguridad
- **Contraseñas robustas:** Mínimo 8 caracteres, mayúsculas, minúsculas, números, símbolos

Base de Datos

- **Usuarios limitados por IP de origen**
- **Solo permisos SELECT, INSERT, UPDATE, DELETE** (no DDL)
- **Sin usuario root remoto**
- **Firewall iptables adicional** en el servidor

FTP

- **Autenticación mediante AD**
- **Carpetas segregadas por departamento**
- **Permisos NTFS específicos**

7.3 Firewall iptables

Políticas por Defecto

```
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT
```

Reglas Críticas de Seguridad

1. Bloquear acceso directo a la base de datos desde DMZ

```
# Solo backends en Red Interna pueden acceder
iptables -A FORWARD -s 192.168.60.41 -d 192.168.60.50 -p tcp --dport 3306 -j ACCEPT
iptables -A FORWARD -s 192.168.60.42 -d 192.168.60.50 -p tcp --dport 3306 -j ACCEPT
# Todo lo demás bloqueado por política DROP
```

2. Balanceador solo puede comunicar con backends

```
iptables -A FORWARD -s 192.168.50.30 -d 192.168.50.41 -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -s 192.168.50.30 -d 192.168.50.42 -p tcp --dport 80 -j ACCEPT
```

3. Logging de intentos bloqueados

```
iptables -A FORWARD -j LOG --log-prefix "FW-BLOCKED: " --log-level 4
iptables -A INPUT -j LOG --log-prefix "FW-INPUT-BLOCKED: " --log-level 4
```

Verificación de Logs

```
# Ver intentos bloqueados
sudo grep FW-BLOCKED /var/log/syslog
```

```
# Monitorización en tiempo real
sudo tail -f /var/log/syslog | grep FW-BLOCKED
```

7.4 Buenas Prácticas Implementadas

Principio de mínimo privilegio

- Los usuarios solo tienen acceso a lo estrictamente necesario

Segmentación de red

- Separación DMZ / Red Interna

Contraseñas robustas

- Política de contraseñas fuertes en AD
- Contraseñas de base de datos complejas

✅ **Logging y auditoría**

- Logs del firewall
- Logs de acceso web (Nginx)
- Logs de MySQL
- Logs de Active Directory

Principio de defensa en profundidad

- Firewall de red (iptables)
- Firewall de aplicación (permisos MySQL)
- Autenticación (AD, usuarios BD)
- Autorización (grupos de AD, permisos NTFS)

---

8. INSTALACIÓN Y CONFIGURACIÓN

8.1 Requisitos Previos

Hardware (mínimo)

- **CPU:** 4 núcleos
- **RAM:** 8 GB
- **Disco:** 50 GB libres
- **Red:** Adaptador Ethernet

Software

- **VirtualBox** 6.1 o superior
- **Vagrant** 2.2 o superior
- **Windows Server 2019/2022** (para el servidor AD)

- **Windows 10/11 Pro** (para clientes)

8.2 Instalación Paso a Paso

PASO 1: Preparar el entorno

```
# Crear directorio del proyecto
mkdir verificanet
cd verificanet

# Copiar archivos
# - Vagrantfile
# - provision/firewall.sh
# - provision/web-frontend.sh
# - provision/backend.sh
# - provision/database.sh
```

PASO 2: Levantar la infraestructura Linux

```
# Levantar todas las VMs (puede tardar 15-20 minutos)
vagrant up

# Verificar que están funcionando
vagrant status

# Verificar conectividad
vagrant ssh firewall
ping 192.168.50.30
ping 192.168.60.50
exit
```

PASO 3: Verificar el balanceador

Desde tu PC, abre un navegador:

http://localhost:8080

Deberías ver la página del balanceador.

Probar la API:

http://localhost:8080/api.php?action=health

Respuesta esperada:

{"status":"ok","database":"connected"}

PASO 4: Configurar Windows Server

1. **Instalar Windows Server 2019/2022**
    - Configurar IP estática: 192.168.50.10
    - Máscara: 255.255.255.0
    - Gateway: 192.168.50.1
    - DNS: 127.0.0.1 (tras instalar AD)
2. **Ejecutar scripts PowerShell en orden:**

```
# 1. Instalar Active Directory
.\Sin_título1.ps1
# El servidor se reiniciará

# 2. Crear OUs
.\OUS.ps1

# 3. Crear usuarios y grupos
.\Usuarios-grupos.ps1

# 4. Configurar DNS
.\DNS.ps1

# 5. Configurar DHCP
.\DHCP.ps1

# 6. Configurar FTP
.\ftp.ps1
```

PASO 5: Unir clientes Windows al dominio

En el cliente Windows:

1. Configurar IP estática o DHCP
2. DNS: 192.168.50.10
3. Unir al dominio:

```
Add-Computer -DomainName "verificanet.local" -Restart
```

Credenciales:

- Usuario: `adminvnet`
- Contraseña: `Verific@2024!`

PASO 6: Configurar el panel Python

En Windows Server o cliente con permisos:

```
# Instalar Python 3.12
# Descargar desde https://www.python.org/downloads/
```

```
# Configurar SSH sin contraseña
ssh-keygen -t rsa -b 4096
type $env:USERPROFILE\.ssh\id_rsa.pub | ssh vagrant@192.168.50.30 "cat >>
~/.ssh/authorized_keys"

# Ejecutar el panel
python admin_logs2.py
```

8.3 Verificación Final

Comandos de Verificación

```
# Desde el firewall
vagrant ssh firewall
sudo iptables -L -n | grep ACCEPT
curl http://192.168.50.30/health

# Desde el servidor web
vagrant ssh web
curl http://192.168.50.41/api.php?action=health
curl http://192.168.50.42/api.php?action=health

# Desde backend1
vagrant ssh backend1
mysql -h 192.168.60.50 -u verificanet_user -p'Verific@2024!' verificanet_servicios -e "SELECT
COUNT(*) FROM incidencias;"

# Desde Windows Server
Get-ADUser -Filter * | Select Name
Get-DhcpServerv4Lease -ScopeId 192.168.50.0
nslookup web.verificanet.local
```

---

9. APÉNDICES

9.1 Tabla de IPs Completa

Host	IP DMZ	IP Interna	Función
Firewall	192.168.50.1	192.168.60.1	Router/Firewall/NAT
Windows Server	192.168.50.10	-	AD/DNS/DHCP/FTP
Clientes Windows	192.168.50.21-200	-	Estaciones de trabajo
Web (Nginx)	192.168.50.30	-	Balanceador HTTP
Backend1	192.168.50.41	192.168.60.41	Servidor API

Backend2	192.168.50.42	192.168.60.42	Servidor API
Database	-	192.168.60.50	MySQL/MariaDB

9.2 Credenciales del Sistema

Base de Datos

- **Usuario:** verificanet_user@192.168.60.41
- **Contraseña:** Verific@2024!

Active Directory (usuarios empleados)

- **Patrón:** [nombre]@verificanet.local
- **Contraseña:** Verific@2024!
- **Ejemplos:**
  - mgonzalez / Verific@2024!
  - clopez / Verific@2024!

SSH (servidores Linux)

- **Usuario:** vagrant
- **Contraseña:** vagrant
- **Nota:** Configurar claves SSH para acceso sin contraseña

Aplicación Web (usuarios de prueba)

- admin / admin123
- mgonzalez / maria123
- cliente1 / cliente123

9.3 Puertos de Red

Puerto	Protocolo	Servicio	Servidor
22	TCP	SSH	Todos los Linux
53	UDP/TCP	DNS	Windows Server
67-68	UDP	DHCP	Windows Server
21	TCP	FTP	Windows Server
80	TCP	HTTP	Web, Backends
3306	TCP	MySQL	Database
8080	TCP	Port Forwarding	Web (desde host)

9.4 Comandos Útiles

Vagrant

```
vagrant up              # Levantar todas las VMs
vagrant up firewall     # Levantar solo firewall
vagrant halt            # Detener todas las VMs
vagrant reload web      # Reiniciar VM web
vagrant ssh backend1    # Conectar por SSH
vagrant destroy -f      # Destruir todas las VMs
vagrant status          # Ver estado
vagrant provision web   # Re-ejecutar aprovisionamiento
```

iptables

```
sudo iptables -L -n -v                  # Ver reglas
sudo iptables -t nat -L -n -v           # Ver reglas NAT
sudo iptables -F                        # Limpiar todas las reglas
sudo netfilter-persistent save          # Guardar reglas
sudo netfilter-persistent reload        # Recargar reglas
```

Nginx

```
sudo systemctl status nginx             # Ver estado
sudo systemctl restart nginx            # Reiniciar
sudo nginx -t                           # Verificar configuración
sudo tail -f /var/log/nginx/access.log  # Ver accesos
sudo tail -f /var/log/nginx/error.log   # Ver errores
```

MySQL

```
sudo systemctl status mariadb           # Ver estado
mysql -u root -p                        # Conectar como root local
mysql -h 192.168.60.50 -u verificanet_user -p  # Conectar remotamente
SHOW DATABASES;                         # Listar bases de datos
USE verificanet_servicios;              # Seleccionar BD
SHOW TABLES;                            # Listar tablas
SELECT * FROM incidencias;              # Consultar datos
```

Active Directory (PowerShell)

```
Get-ADUser -Filter *                    # Listar usuarios
Get-ADGroup -Filter *                   # Listar grupos
Get-ADComputer -Filter *                # Listar equipos
Get-ADOrganizationalUnit -Filter *      # Listar OUs
Get-DnsServerZone                       # Ver zonas DNS
Get-DhcpServerv4Scope                   # Ver ámbitos DHCP
Test-ComputerSecureChannel -Verbose     # Verificar unión al dominio
```

9.5 Solución de Problemas

Problema: No se puede acceder a la web (localhost:8080)

Solución:

```
# Verificar que web está levantada
```

```
vagrant status

# Verificar Nginx
vagrant ssh web
sudo systemctl status nginx
curl http://localhost

# Verificar reglas de firewall
vagrant ssh firewall
sudo iptables -L -n | grep 192.168.50.30
```

Problema: API devuelve error de conexión a BD

Solución:

```
# Verificar que database está arrancada
vagrant ssh database
sudo systemctl status mariadb

# Probar conexión desde backend
vagrant ssh backend1
mysql -h 192.168.60.50 -u verificanet_user -p'Verific@2024!' verificanet_servicios

# Verificar firewall
vagrant ssh firewall
sudo iptables -L -n | grep 192.168.60.50
```

Problema: Cliente Windows no se puede unir al dominio

Solución:

```
# Verificar conectividad con DC
ping 192.168.50.10

# Verificar DNS
nslookup verificanet.local

# Configurar DNS manualmente
Set-DnsClientServerAddress -InterfaceAlias "Ethernet" -ServerAddresses 192.168.50.10

# Reintentar unión al dominio
Add-Computer -DomainName "verificanet.local" -Credential (Get-Credential) -Restart
```

Problema: Panel Python no conecta por SSH

Solución:

```
# Verificar conectividad
ping 192.168.50.30

# Probar SSH manualmente
```

```
ssh vagrant@192.168.50.30

# Verificar clave SSH
ls $env:USERPROFILE\.ssh\
cat $env:USERPROFILE\.ssh\id_rsa.pub

# Reenviar clave pública
type $env:USERPROFILE\.ssh\id_rsa.pub | ssh vagrant@192.168.50.30 "cat >>
~/.ssh/authorized_keys"
```

CONCLUSIÓN

Este manual técnico documenta la implementación completa del proyecto **Verificanet**, un sistema de gestión de incidencias con:

Alta disponibilidad mediante balanceo de carga
Seguridad robusta con DMZ y segmentación de red
Gestión centralizada con Active Directory
Automatización mediante Vagrant y scripts
Monitorización con panel de administración Python

El sistema está diseñado para ser:

- **Escalable** - Fácil de añadir más backends o servicios
- **Seguro** - Políticas de firewall estrictas y control de acceso
- **Mantenible** - Scripts documentados y automatización completa
- **Profesional** - Arquitectura estándar de la industria